
WireGuard



Main concepts (1/3)

▷ Simplicity

- ♦ Extremely small and easy to understand
- ♦ Its entire codebase is only a few thousand lines
 - Much smaller than VPN protocols like OpenVPN or IPsec
 - Easier to audit, maintain, and secure

▷ Fixed, modern set of crypto primitives

- ♦ Curve25519 for key exchange (EC Diffie-Hellman)
- ♦ ChaCha20 for encryption (stream cipher)
- ♦ Poly1305 for MACs
- ♦ BLAKE2s for hashing (faster than SHA-2 and SHA-3)
- ♦ No cryptographic options, less misconfiguration risks

Main concepts (2/3)

▷ Built to be fast

- ♦ Highly optimized cryptography
- ♦ Kernel-level implementation on Linux
- ♦ Minimal protocol overhead
- ♦ Extremely high throughput and low latency compared to traditional VPNs

▷ Ease of configuration

- ♦ Simple key pairs (like SSH) instead of certificates
- ♦ A basic config usually has just:
 - Private key
 - Public key
 - Allowed IP ranges
 - Endpoint

Main concepts (3/3)

▷ Roaming support

- ◆ Handles network changes gracefully
- ◆ No tunnel renegotiation when a client's IP changes

▷ Peer-to-peer approach

- ◆ Each WireGuard node is a “peer”
- ◆ Any peer can initiate communication as long as it knows the other peer's public key
 - And real IP address

▷ Integrated into the Linux Kernel

- ◆ Also available on Windows, MacOS, iOS, Android, and BSD systems

WireGuard: tunnel configurations

```
[Interface]
Address = 10.0.0.1/24

[Peer]
PublicKey = peerPublicKey
Endpoint = 198.51.100.7:51820
AllowedIPs = 10.0.0.2/32
```

▷ Key pair setup

- ◆ Each peer generates:
 - Private key (kept secret)
 - Public key (shared with other peers)
- ◆ These keys are the basis of authentication and encryption

▷ Exchange of public keys

- ◆ Peers securely share their public keys out-of-band
- ◆ No certification infrastructure

▷ Interface configuration

- ◆ Each peer creates a virtual network interface (e.g., wg0)
- ◆ This interface gets an IP address inside the VPN (private IP address)

▷ Allowed IPs mapping (private network address range)

- ◆ Each peer defines Allowed IPs, which define:
 - What traffic should go into the tunnel
 - Which peer a packet is routed to
- ◆ This acts like both a routing table and an access control list

WireGuard: tunnel creation

▷ Handshake

- ◆ Compact version of the NoiseIK handshake
 - To setup session keys and authenticate peers
 - Very fast (single round trip)
- ◆ Ensures mutual authentication
 - Using long-term asymmetric key pairs
- ◆ Establishes shared symmetric session keys
 - Using ephemeral asymmetric key pairs and EC DH
 - Perfect forward secrecy

▷ Handshakes only occur when needed

- ◆ When new session keys are necessary

WireGuard: tunnel operation

- ▷ Packets entering the **wg** interface are encrypted
 - ◆ Using ChaCha20-Poly1305
- ▷ Encrypted packets are wrapped in UDP
 - ◆ and sent to the peer's UDP/IP endpoint
 - ◆ They also contain a session index and a packet counter
- ▷ On arrival, they are unwrapped, validated and decrypted
 - ◆ then forwarded inside the peer's network stack
 - ◆ The IPs used upon forwarding are always the VPN IPs
- ▷ Session keys rotate automatically every few minutes
 - ◆ Running the NoiseIK again

WireGuard: tunnel operation

▷ Silent when idle

- ◆ No keep-alives unless NAT traversal or roaming requires it

▷ Roaming & endpoint updating

- ◆ If a peer's IP changes, the next authenticated packet updates the peer's endpoint
- ◆ The tunnel continues without reconfiguration

WireGuard general usage

- ▷ It may require a third-party key management infrastructure
 - ♦ e.g. Tailscale & tailnets
- ▷ Is not easy to deploy for being used by thousands of users for a single VPN gateway
 - ♦ User-IP mapping
 - ♦ Key pair distribution